

Divide and conquer: General method, applications- Binary Search, Finding maximum and minimum, Quick sort, merge sort, Strassen's matrix multiplication.

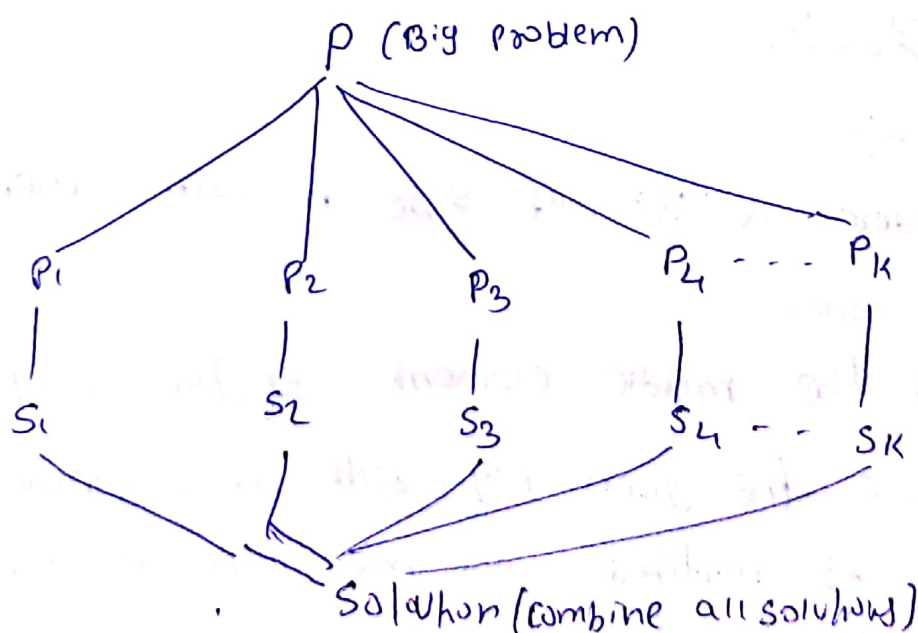
General method:

The divide-and-conquer approach involves three steps at each level of recursion.

1. Divide: Break the problem into several sub problems that are similar to the original problem but smaller in size

2. Conquer: solve the sub problems recursively. If the subproblem size are small enough, just solve the sub problem in a straight forward manner

3. Combine: combine these solutions to create a solution to the original problem.



Algorithm:

DAC(P)

{

if (small(P))

{

SIP);

}

else

{

divide P into $P_1, P_2, P_3 \dots P_k$ Apply DAC(P_1), DAC(P_2) ... - DAC(P_k)combine (DAC(P_1), DAC(P_2) ... - DAC(P_k))

{

}

Time complexity General:

$$T(n) = \begin{cases} T(1) & n \text{ is small} \\ AT(n/b) + f(n) & \text{when } n \text{ is big} \end{cases}$$

Binary Search:-Procedure:-

- consider a list of size 'n' with elements in ascending order
- Find the middle element of the list
- compare the given key with mid element
- If key is matched with mid element then return

location that key is found

→ If key is less than mid element then make high as mid-1 and repeat above steps until the element is found

→ if key is greater than mid element then make low as mid+1 and repeat above steps until the element is found

→ Even doing all if element is not found i.e. when low is greater than high and return element not found.

Algorithm :-

R BinarySearch (l, h, Key) ——— $T(n)$

```
{  
  if (l == h)  
  {  
    if (A[l] == Key)  
      return l;  
    else  
      return 0;  
  }  
}
```

```
else  
{  
  mid = (l+h)/2;  
  if (Key == A[mid])  
    return mid;  
}
```

(4)

if (key < A[mid])

return RBinarySearch(l, mid-1, key)

else

return RBinarySearch(mid+1, h, key)

} - $T(n/2)$

}

Time complexity:

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + 1 & n > 1 \end{cases}$$

$$T(n) = T(n/2) + 1 \quad \text{--- (1)}$$

$$T(n/2) = T(n/2^2) + 1$$

substitute $T(n/2)$ in (1)

$$T(n) = [T(n/2^2) + 1] + 1$$

$$T(n) = T(n/2^2) + 2 \quad \text{--- (2)}$$

$$T(n/2^2) = T(n/2^3) + 1$$

substitute $T(n/2^2)$ in (2)

$$\begin{aligned} T(n) &= [T(n/2^3) + 1] + 2 \\ &= T(n/2^3) + 3 \end{aligned}$$

After k iterations

$$T(n) = T(n/2^k) + k$$

$$\frac{n}{2^k} = 1 \quad (\text{Base solution } T(n) = 1 \text{ when } n = 1)$$

$$n = 2^k$$

$$k = \log_2 n$$

$$T(n) = \log n + 1$$

5

Time complexity = $O(\log n)$

for Binary search using Divide and Conquer the

Time complexity of worst case

Best case

Average case

$$O(\log n)$$

Example:

Find the element ~~24~~ ^{Key: 59} in the following array

5	9	17	23	25	45	59	63	71	89
0	1	2	3	4	5	6	7	8	9

Find mid point $\lceil (0+9)/2 \rceil = 5$

mid Element: 45 - 5th location.

low	high	mid	
0	9	5	^{key} (59 > 45)
6	9	7	^{key} (59 < 63)
6	6	6	^{key} (59 == 59)

element found

Key = 24

low	high	mid		low	high	mid	
0	9	5	^{key} (24 < 45)	3	3	3	$24 > 23$
0	4	2	(24 > 17)				
0	4	4	(24 < 25)				
3	3	3	(24 > 23)				

element
not found



Finding maximum and minimum:

(6)

→ The problem is to find the maximum and minimum items in a set of 'n' elements.

→ In analysing the time complexity of this algorithm, we once concentrate on the element comparisons.

Algorithm:

DAC MAX_MIN(A, i, j, max, min) — $T(n)$

~~int~~ mid = ~~(i+j)/2~~

{
if (i == j)

max = min = A[i]

else if (i == j-1)

{
if (A[i] < A[j])

max = A[j], min = A[i];

else

max = A[i], min = A[j];

}
else
{ mid = (i+j)/2

DAC MAX_MIN(A, i, mid, max, min) — $T(n/2)$

DAC MAX_MIN(A, mid+1, j, max, min) — $T(n/2)$

if (max < max1) then

max ← max1

if (min > min1) then

min ← min1

}

Time complexity:

⑦

$$T(n) = \begin{cases} 0 & n=1 \\ 1 & n=2 \\ 2T(n/2)+2 & n>2 \end{cases}$$

$$T(n) = 2T(n/2) + 2 \quad \text{--- ①}$$

$$T(n/2) = 2T(n/4) + 2$$

substitute $T(n/2)$ in ①

$$\begin{aligned} T(n) &= 2[2T(n/4) + 2] + 2 \\ &= 4T(n/4) + 4 + 2 \end{aligned}$$

After K -iterations

$$T(n) = 2^K T(n/2^K) + \dots$$

$$= 2^K - 1 + 2^K - 2$$

$$= n/2 + n - 2$$

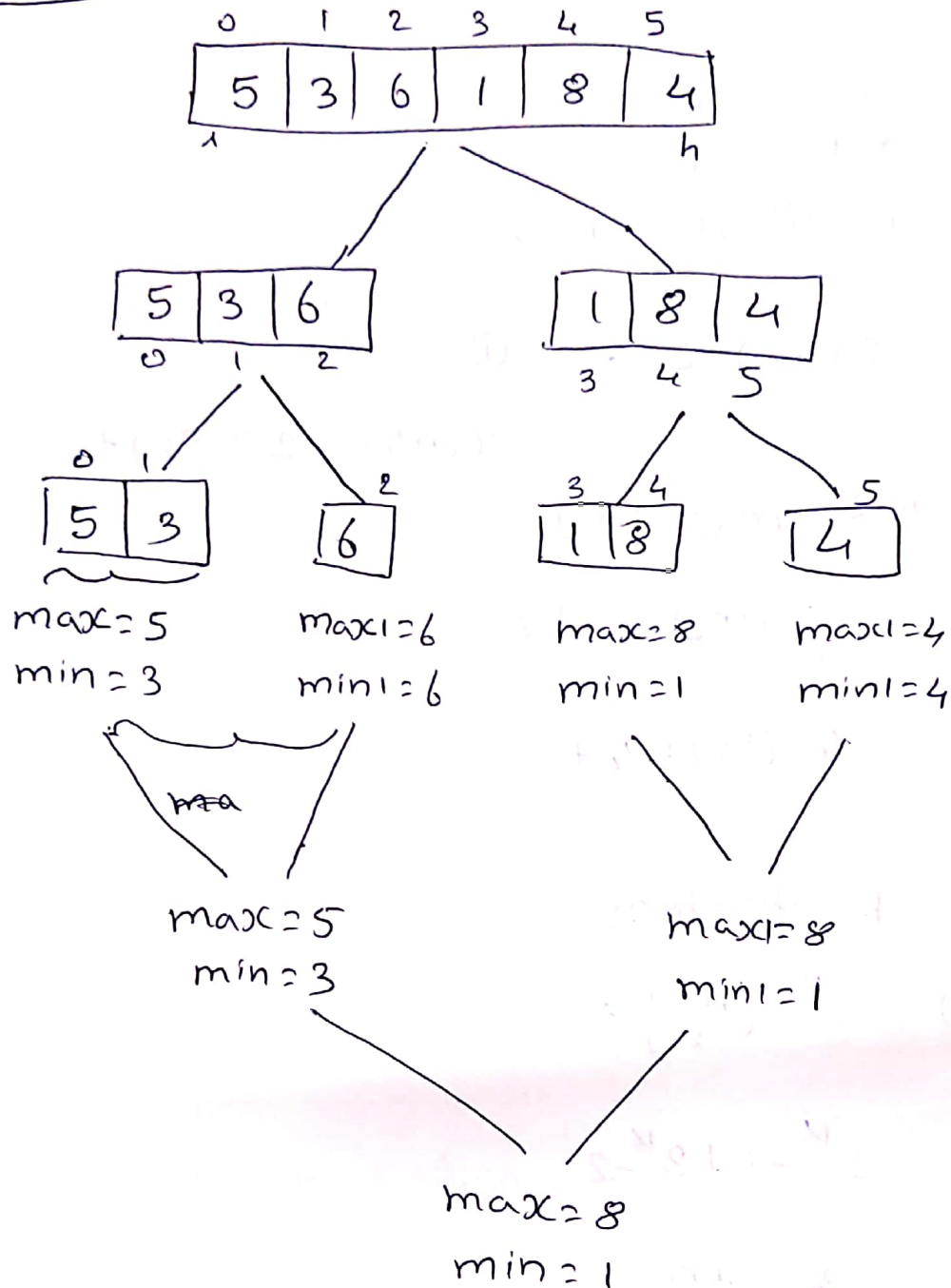
$$= (n + 2n)/2 - 2$$

$$T(n) = (3n/2) - 2$$

Time complexity $O(n)$

example:

8



First we divide Array in to small sub problem by using mid point and then ~~can~~ find the solution to small sub problem and combine the solution of small sub problem to get the solution of Big problem.

Quick sort:-

- Divide and conquer principle is used in quick sort
- Quick sort works by partitioning a given array $A[p \dots r]$ into two non-empty sub arrays $A[p \dots q]$ and $A[q+1 \dots r]$
- It chooses the element as a pivot element, a lower bound is first index and upper bound is last index
- Based on the pivot element chosen the array is partitioned such that all the greater elements are moved to right side of pivot and all smaller elements are moved to left side of pivot.
- Each partition is internally sorted recursively

Algorithm:-

QuickSort(l, h) — $T(n)$

{

if ($l < h$) — 1

{

$j = \text{partition}(l, h);$ — n

QuickSort(l, j); — $T(n/2)$

QuickSort($j+1, h$); — $T(n/2)$

}

(partition occurred at first)
Assuming pivot element
taken as first element

$T(0) \rightarrow$ Assuming pivot
element taken
at middle

(partition occurs at
middle)

```

Partition(l, h)
{
    pivot = A[l]
    i = l, j = h
    while (i < j)
    {
        do
        {
            i++;
        } while (A[i] ≤ pivot);
        do
        {
            j--;
        } while (A[j] ≥ pivot);
        if (i < j)
            swap(A[i], A[j]);
    }
    swap(A[l], A[j]);
    return j;
}

```

Time Analysis:-

Worst case:-

- Running time depends on which element is chosen as key or pivot
- The worst case occurs when the array is partitioned into one subarray with $n-1$ elements and the other with 0 elements

→ In this case

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned}$$

$$T(n) = \Theta(n^2)$$

Best case:-

→ occurs when partition produces subproblems each of size $n/2$

→ $T(n) = 2T(n/2) + \Theta(n)$ refer notes 1st unit

$$T(n) = \Theta(n \log n)$$

Average case:-

→ Average case is much closer to best case.

→ if suppose the partitioning algorithm produces a 9:1 proportional split the recurrence will be

$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$

$$T(n) = \Theta(n \log n)$$

Example:

②

Consider the following unsorted list of elements

A	5	3	8	1	4	6	2	7	∞
---	---	---	---	---	---	---	---	---	----------

Define pivot, left & right set $\text{Pivot} = A[0]$, $\text{left} = A[0]$ $\text{right} = A[8]$
Increment left and decrement right
 $i =$ $j =$

Compare $A[\text{left}]$ with $A[\text{pivot}]$. If $A[\text{left}]$ is greater than $A[\text{pivot}]$ then stop left otherwise move left to the next

Compare $A[\text{right}]$ with $A[\text{pivot}]$. If $A[\text{right}]$ is smaller than $A[\text{pivot}]$ then stop right otherwise move right to the previous

Repeat the same until $\text{left} \geq \text{right}$

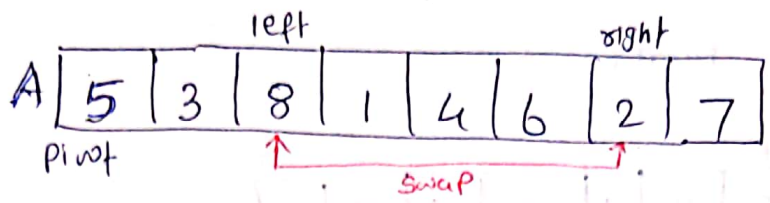
If both left & right are stopped but $\text{left} < \text{right}$ then swap $A[\text{left}]$ with $A[\text{right}]$ and continue the process

If $\text{left} \geq \text{right}$ then swap $A[\text{pivot}]$ with $A[\text{right}]$

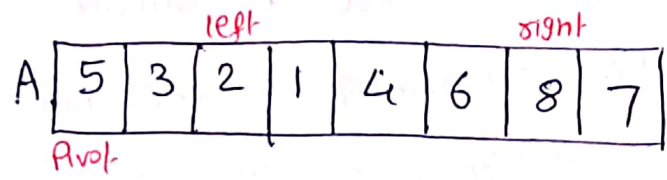
		left						right	
A	5	3	8	1	4	6	2	7	
	pivot								

compare $A[\text{left}] < A[\text{pivot}]$ as it is true increment left by one and repeat the same, left will stop at 8.

compare $A[\text{right}] > A[\text{pivot}]$ as it is true decrement right by one and repeat the same, right will stop at 2

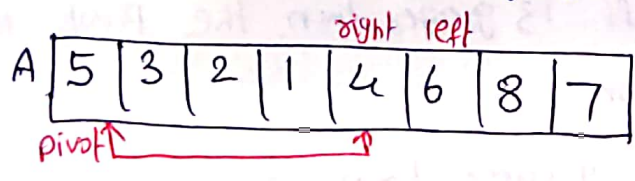


Here left & right both are stopped and left is not greater than right so, we need to swap $A[\text{left}]$ and $A[\text{right}]$

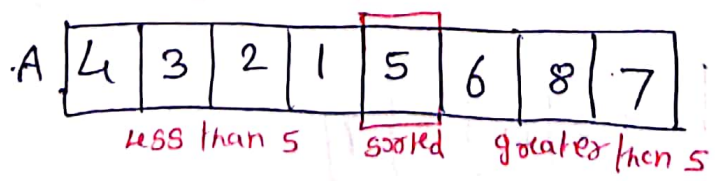


compare $A[\text{left}] < A[\text{pivot}]$ as it is true increment left by one and repeat the same, left will stop at '6'

compare $A[\text{right}] > A[\text{pivot}]$ as it is true decrement right by one and repeat the same, right will stop at 4

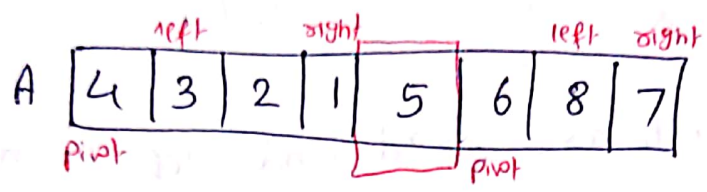


Here left & right both are stopped and left is greater than right so we need to swap $A[\text{pivot}]$ and $A[\text{right}]$

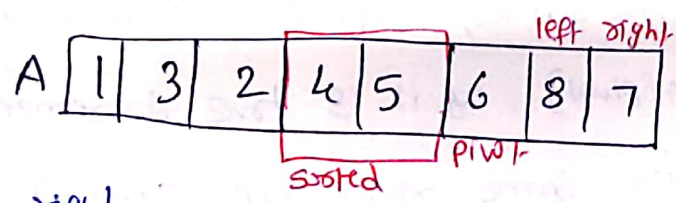


Here we can observe that all the numbers to the left side of 5 are smaller and right side are greater. That means 5 is placed in correct position.

Repeat the same process on the left sublist and right sublist to the number 5



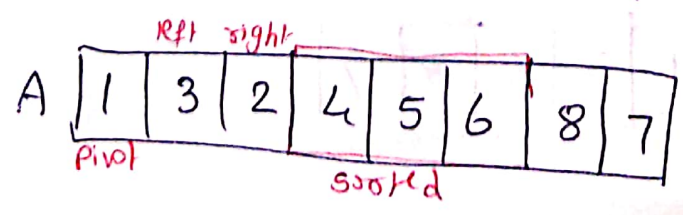
In the left sublist as there are no smaller number than the pivot left will keep on moving to the next and stop at last number. As the $A[\text{right}]$ is smaller, right stops at same position. Now left and right both are equal so we swap pivot with right



In the right sublist left is greater than the pivot, left will stop at same position.

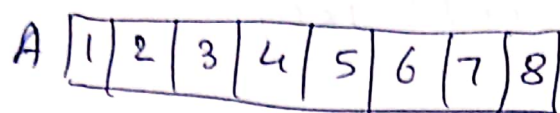
As the $A[\text{right}]$ is greater than $A[\text{pivot}]$, right moves towards left and stops at pivot number position

Now $\text{left} > \text{right}$ so, we swap pivot with right (6 is swapped by itself)



Repeat the same recursively on both left and right sublists until all the numbers are sorted

The final sorted list will be as follows



merge sort:

→ merge sort is an example of divide and conquer algorithm

→ It is based on the idea of breaking down a list into several sub-list until each sublist consists of a single element

→ merging those sublists in a manner that results into a sorted list.

Algorithm:-

```

mergesort(l, h)  → T(n)
{
    if(l < h)
    {
        mid = (l+h)/2
        mergesort(l, mid);  → T(n/2)
        mergesort(mid+1, h);  → T(n/2)
        merge(l, mid, h);  → n
    }
}

```

merge(l, mid, h)

```

{
    i = l; j = 1; k = l;
    while (i ≤ m && j ≤ n)
    {
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];
    }
    for (; i ≤ m; i++)
        C[k++] = A[i];
    for (; j ≤ n; j++)
        C[k++] = B[j];
}

```

m = number of elements
in first array

n = number of elements
in second array

Analysis:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

$$T(n) = 2T(n/2) + n \rightarrow \textcircled{1}$$

Substitute $T(n/2)$ in $\textcircled{1}$

$$T(n/2) = 2T(n/4) + n/2$$

$$T(n) = 2[2T(n/2) + n/2] + n$$

$$= 4T(n/2) + n + n$$

$$T(n) = 2^2 T(n/2^2) + 2n$$

After k iterations

$$T(n) = 2^k T(n/2^k) + kn$$

$$\text{Assume } \frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log_2 n$$

$$T(n) = n T(1) + n \log n$$

$$T(n) = n + n \log n$$

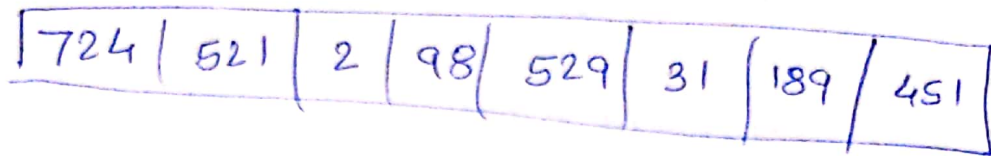
$$T(n) = \Theta(n \log n)$$

Note.

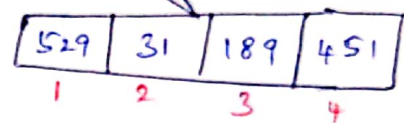
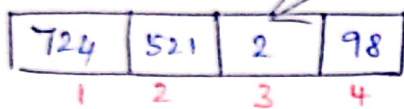
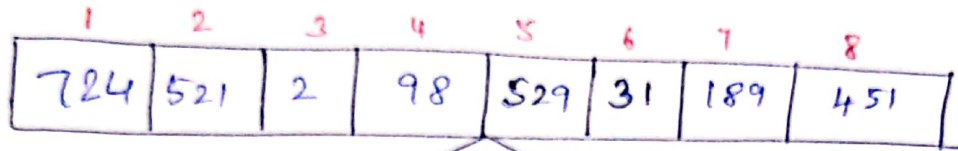
The Best, Average, worst case of merge sort is
 $n \log n \therefore \Theta(n \log n)$

example:

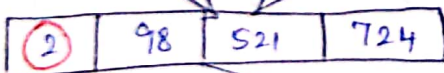
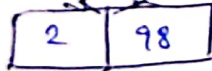
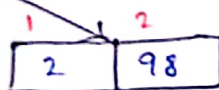
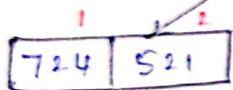
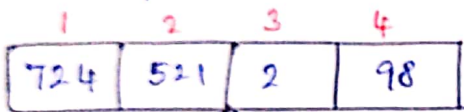
18



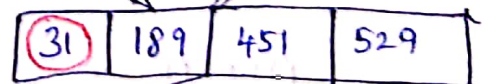
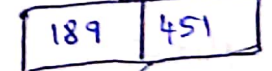
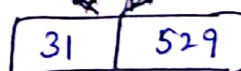
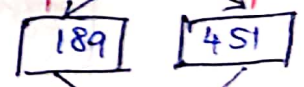
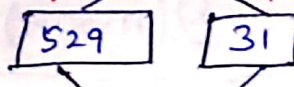
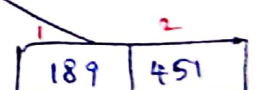
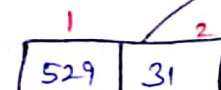
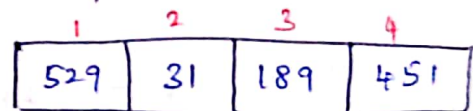
Step 1: Split the selected array based on mid element



Select the left subarray and split

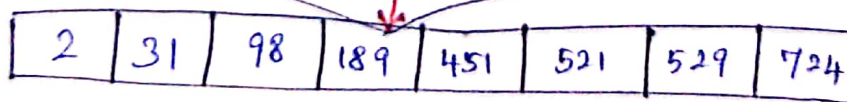


Select the right subarray and split.



Split

Merge



Strassen's matrix multiplication

Let us consider two matrices x and y and calculate the resultant matrix z by multiplying x and y

iterative method:

First we will see iterative (naive) method and its time complexity. Here we are calculating $z = x * y$ using iterative method,

Algorithm $mm(x, y, z)$

for $i = 1$ to p do

for $j = 1$ to r do

$z[i, j] = 0$

for $k = 1$ to q do

$z[i, j] = z[i, j] + x[i, k] * y[k, j]$

Complexity:

There are three 'for' loops in this algorithm and each one is independent of each other. Hence the algorithm takes $O(n^3)$.

Strassen's matrix multiplication:

→ In this context, using Strassen's matrix multiplication algorithm, the time consumption can be improved a little bit

→ Strassen's matrix multiplication can be performed only on square matrices where ' n ' is a power of 2
order of both the matrices are $n \times n$

→ Divide A, B and C into four $n/2 \times n/2$ matrices as represented below

ex:-

$$n=2 \quad A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \Rightarrow C = \begin{bmatrix} c_{11} & c_{12} \\ c_{13} & c_{14} \end{bmatrix}$$

when

$n=4$

$$A = \begin{bmatrix} \begin{matrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{matrix} & \begin{matrix} a_{13} & a_{14} \\ a_{23} & a_{24} \end{matrix} \\ \begin{matrix} a_{31} & a_{32} \\ a_{41} & a_{42} \end{matrix} & \begin{matrix} a_{33} & a_{34} \\ a_{43} & a_{44} \end{matrix} \end{bmatrix} \quad \begin{matrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{matrix} \quad \times \quad B = \begin{bmatrix} \begin{matrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{matrix} & \begin{matrix} b_{13} & b_{14} \\ b_{23} & b_{24} \end{matrix} \\ \begin{matrix} b_{31} & b_{32} \\ b_{41} & b_{42} \end{matrix} & \begin{matrix} b_{33} & b_{34} \\ b_{43} & b_{44} \end{matrix} \end{bmatrix} \quad \begin{matrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{matrix}$$

Algorithm:-

Algorithm $mm(A, B, n)$

{

if ($n \leq 2$)

{

$$c_{11} = A_{11} * B_{11} + A_{12} * B_{21}$$

$$c_{12} = A_{11} * B_{12} + A_{12} * B_{22}$$

$$c_{21} = A_{21} * B_{11} + A_{22} * B_{21}$$

$$c_{22} = A_{21} * B_{12} + A_{22} * B_{22}$$

}

else

$$mid = n/2$$

$$\begin{aligned}
 &mm(A_{11}, B_{11}, n/2) + mm(A_{12}, B_{21}, n/2) \\
 &mm(A_{21}, B_{12}, n/2) + mm(A_{12}, B_{22}, n/2) \\
 &mm(A_{21}, B_{11}, n/2) + mm(A_{22}, B_{21}, n/2) \\
 &mm(A_{21}, B_{12}, n/2) + mm(A_{22}, B_{22}, n/2)
 \end{aligned}$$

Computing each entry in the product takes n multiplications and there are n^2 entries for a total of $O(n^3) \rightarrow T(n) = 8T(n/2) + n^2$ solving we get $O(n^3)$

- $\rightarrow$ Strassen's devised a better method which has the same basic method as multiplication of long integers
- $\rightarrow$ The main idea is to save one multiplication on a small problem and then use a recursion.

Strassen's Algorithm for matrix multiplication involves

Step 1:

$$\begin{aligned}
 P &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
 Q &= (A_{21} + A_{22})B_{11} \\
 R &= A_{11}(B_{12} - B_{22}) \\
 S &= A_{22}(B_{21} - B_{11}) \\
 T &= (A_{11} + A_{12})B_{22} \\
 U &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
 V &= (A_{12} - A_{22})(B_{21} + B_{22})
 \end{aligned}$$

Step 2:

$$\begin{aligned}
 C_{11} &= P + S - T + V \\
 C_{12} &= R + T \\
 C_{21} &= Q + S \\
 C_{22} &= P + R - Q + U
 \end{aligned}$$

Analysis:-

→ It is therefore possible to multiply two 2×2 matrices using only seven scalar multiplication

→ Let $T(n)$ be the time needed to multiply two $n \times n$ matrices by recursive use of equation

$$T(n) = 7T(n/2) + f(n)$$

$$\text{where } f(n) = O(n^2)$$

→ If we compute above using Back substitution method or by master theorem method we get

$$T(n) = O(n^{2.81})$$

so, Strassen's matrix multiplication reduced the time complexity from $O(n^3)$ to $O(n^{2.81})$

Short Questions:

1. Derive the worst-case complexity of the Quick sort algorithm.
2. In how many passes does the Quick sort technique sorts the following sequence 3, 27, 4, 11, 45, 39, 2, 16, 56.
3. Explain the basic principle of Divide and Conquer method
4. Write the control abstraction for divide and conquer.
5. What is the significance of Strassen's matrix multiplication?
6. Describe the Algorithm Analysis of Binary Search
7. Write the general algorithm for Divide and Conquer
8. Describe the time complexity of Divide And Conquer in the recurrence form.
9. Determine the number of passes required to search the element 44 in the following list of elements 5,12,17,23,38,44,77,84,90.
10. Why is it necessary to have the auxiliary array $b[low: high]$ in function *Merge*?

Long Questions:

1. What are the advantages and disadvantages of Divide – And – Conquer
2. State the best, average and worst-case complexities of binary search for successful and unsuccessful search.
3. Explain the principle of Binary search technique with an example and also List the drawbacks of it
4. Apply quick sort algorithm to sort the list. E, X, A, M, P, L, E in alphabetical order.
5. Show the result of running Merge sorting technique and quick sorting technique on the sequence 38,27,43,3,9,82,10
6. Given 2 sorted lists of numbers. Write the algorithm to merge them and analyze its time complexity.
7. Write the merge sort algorithm. Find out the best, worst and average cases of this algorithm. Sort the following numbers using merge sort: 10, 12, 1, 5, 18, 28, 38, 39, 2, 4, 7
8. Sort the list of the elements 10,5,7,6,1,4,8,3,2,9 using merge sort algorithm and show its computing time is $O(n \log n)$.
9. Write Divide- And- Conquer recursive Merge sort algorithm
10. Explain quick sort algorithm with an example. Derive its time complexity.
11. Distinguish between Merge sort and quick sort
12. What is stable sorting method? Is Merge sort a stable sorting method?
13. Mention the advantage of Strassen's matrix multiplication. Explain how the Strassen's matrix multiplication works.
14. Write an algorithm for finding the Maximum and Minimum values using Divide and Conquer method and solve for: 22, 13, -5, -8, 15, 60, 17, 31, 47.
15. Explain recursive Binary search algorithm and derive its time complexity.
16. Demonstrate binary search algorithm and find the element $x = 12$ in the following array
 - a. 2, 5, 8, 12, 16, 23, 38, 56, 72, 91.
17. Explain binary search algorithm and find the element $x = 31$ in the following array

- a. 10, 15, 18, 26, 27, 31, 38, 45, 59.
- 18. Sort the following array in ascending order using quick sort algorithm.
 - a) 5, 3, 8, 9, 1, 7, 0, 2, 6, 4
 - b) 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9
 - c) 9, 7, 5, 11, 12, 2, 14, 3, 10, 6
- 19. Explain partition exchange sort algorithm and trace this algorithm for $n=8$ elements:
 - a. 24, 12, 35, 23, 45, 34, 20, 48
- 20. Show the result of running Merge sorting technique on the sequence 38, 27, 43, 3, 9, 82, 10 and also derive best, worst and average time complexities of merge sort